

QUESTION PAPER GENERATION SYSTEM

A Major Project report submitted in seventh semester to CUSAT in partial fulfillment of the requirements for the award of degree

BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE & ENGINEERING

Submitted by,

VINAYAK M V (20222567)
ANN TREESA SONEY (20222514)
ASWIN R (20222572)
ABHISHEK P V (20222569)
S ANAMIKA (20222557)

Under the Guidance of

Mrs. Alice Joseph
Project Guide, Division of CSE

Ms. Neethu George
Project Guide, Division of CSE

Dr. Preetha Mathew
Project Coordinator, Division of CSE

Mrs. Bindu P K
Head of Department, Division of CSE



Division of Computer Science & Engineering

COCHIN UNIVERSITY COLLEGE OF ENGINEERING KUTTANAD
ALAPPUZHA

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY
COCHIN UNIVERSITY COLLEGE OF ENGINEERING KUTTANAD

Pulincunnoo – 688504

Division of Computer Science & Engineering



BONAFIDE CERTIFICATE

This is to certify that the major project entitled “Question Paper Generation System” is a bonafide report of the work done by **Vinayak M V(20222567)**, **Ann Treesa Soney(20222514)**, **Abhishek P V(20222569)**, **S Anamika(20222557)** & **Aswin R(20222572)** towards the partial fulfilment of the requirements of the degree of B.Tech in Computer Science and Engineering of Cochin University of Science and Technology.

Mrs. Alice Josphe
Project Guide, Division of CSE

Ms. Neethu George
Project Guide, Division of CSE

Dr. Preetha Mathew K
Project Coordinator, Division of
CSE

Mrs. Bindu P K
Head of the Department, Division
Of CSE

ACKNOWLEDGEMENT

We are pleased to present the Phase 1 project report for our major project titled “Question Paper Generation System”. We would like to express our sincere gratitude to Dr. Asaletha R, Principal, Cochin University College of Engineering Kuttanad (CUCEK), for providing us with the facilities and support necessary to carry out this phase of our project.

We extend our heartfelt thanks to Mrs. Bindu P K, Associate Professor & Head of the Department of Computer Science and Engineering, for her valuable guidance, motivation, and encouragement throughout the preparation of this report.

We also wish to express our deep appreciation to Dr. Preetha Mathew, Professor & Project Coordinator, Division of Computer Science and Engineering, for her insightful suggestions and continuous supervision, which greatly helped us in completing this phase successfully.

A special note of gratitude goes to our Project Guides, Mrs. Alice Joseph and Ms. Neethu George, Assistant Professors, Division of Computer Science and Engineering, for their constant guidance, timely feedback, and encouragement during the development of this report.

Finally, we thank all the faculty members and staff of the Department of Computer Science and Engineering, CUCEK, for their cooperation and support during this stage of our project.

TABLE OF CONTENTS

SL NO.	CONTENTS	PAGE NO.
1	INTRODUCTION	7
1.1	Purpose	7
1.2	Intended Audience	7
1.3	Scope	8
1.4	Definitions, Acronyms, and Abbreviations	9
2	LITERATURE SURVEY	10
3	DESCRIPTION	12
3.1	User Interfaces	12
3.2	System Interfaces	13
3.3	Constraints, Assumptions, and Dependencies	14
4	TOOLS REQUIRED	16
4.1	React	16
4.2	Syllabus Extraction / Keywording – Scikit-learn (TF-IDF)	16
4.3	Textbook Parsing / Chunking – PyMuPDF (fitz library)	16
4.4	Semantic Topic–Text Mapping – Sentence-Transformers (SBERT)	17
4.5	Question Generation – Hugging Face Transformers (T5 Model)	17
4.6	Bloom’s Level Classification – DistilBERT	18
4.7	Exam Pattern Engine / Assembly – Custom Python Rule Engine	18
4.8	Export / Formatting – python-docx & ReportLab	18
5	USER CHARACTERISTICS	20

6	SYSTEM FEATURES AND REQUIREMENTS	21
6.1	Functional Requirements	21
6.2	System Diagrams	23
6.3	External Interface Requirements	43
6.4	Logical Database Requirements	43
6.5	Non-Functional Requirements	44
7	CONCLUSION	45
8	REFERENCES	46

TABLE OF FIGURES

No.	Name	Page No.
6.2.1.1	System Design Diagram	24
6.2.2.1	Usecase Diagram for Question Paper Creation	36
6.2.2.2	Usecase Diagram for Question Paper Modification	38
6.2.3.1	Sequence Diagram	40
6.2.4.1	Activity Diagram	41
6.2.5.1	Gantt Chart	42

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to define, in detail, the objectives, design considerations, functional requirements, and constraints of the Automated Question Paper Generator (AQPG) system. This document provides a comprehensive understanding of what the system aims to achieve, how it will function, and the expected behavior under various operating conditions.

The Automated Question Paper Generator system is designed to address the challenges faced by educators in preparing academic question papers manually. Traditional question paper design is time-consuming, inconsistent in difficulty distribution, and often lacks a systematic approach to covering all syllabus modules and Bloom's Taxonomy levels. The proposed system automates this process by leveraging Natural Language Processing (NLP) and Machine Learning (ML) models to analyze syllabus documents, extract key topics, identify relevant textbook content, and generate balanced, syllabus-aligned question papers.

This document serves as a contract between the development team and teachers, ensuring a shared understanding of project goals, deliverables, and technical expectations. It will guide the design, development, testing, and validation of the final software product.

1.2 Intended Audience

This document is intended for the following individuals:

- **Project Developers:** To understand system requirements, data flow, and integration among NLP modules such as TF-IDF, SBERT, T5/BART, and DistilBERT.
- **Project Supervisors / Academic Evaluators:** To verify that the system meets educational objectives, adheres to Bloom's Taxonomy principles, and aligns with institutional assessment standards.
- **End Users (Teachers and Academic Coordinators):** To understand system functionality and capabilities for generating question papers efficiently and accurately.
- **Future Maintainers:** To refer to design decisions, dependencies, and constraints for updates or scalability improvements.

This document assumes that the reader has a basic understanding of software systems, machine learning concepts, and educational assessment structures.

1.3 Scope

The Automated Question Paper Generator (AQPG) aims to build a self-contained application capable of generating complete academic question papers automatically from a given syllabus and set of reference textbooks.

The system's major objectives include:

- **Automated Topic Extraction:** Using TF-IDF to extract keywords and important concepts from syllabus documents.
- **Semantic Mapping:** Applying Sentence-BERT (SBERT) [3] to locate and extract relevant text segments from uploaded textbooks corresponding to each topic.
- **Question Generation:** Employing fine-tuned NLP models such as T5 [4] or BART [5] to generate meaningful questions from textbook passages.
- **Cognitive Level Classification:** Utilizing DistilBERT [6] to categorize each question according to Bloom's Taxonomy levels — Remember, Understand, Apply, Analyze, Evaluate, and Create [1].
- **Question Paper Assembly:** Using a rule-based Exam Pattern Engine to organize questions according to teacher-defined patterns (e.g., number of marks, sections, OR logic, and module distribution) [8].
- **Paper Export:** Generating the final paper in DOCX or PDF format for easy review, editing, and printing.

The system's scope includes teachers, universities, and educational institutions seeking a semi-automated or fully automated solution for question paper creation. It focuses on ensuring syllabus completeness, balanced difficulty, and cognitive diversity — thereby enhancing both fairness and academic rigor in assessments.

Out of scope:

- Evaluation or grading of student answers.
- Integration with external databases or online testing systems.
- Handling of scanned handwritten materials.

1.4 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
TF-IDF	Term Frequency–Inverse Document Frequency — an algorithm for identifying key terms in a document.
SBERT	Sentence-BERT — an NLP model used to compute semantic similarity between sentences or paragraphs.
T5	Text-To-Text Transfer Transformer — an NLP model that treats all text tasks as text generation problems.
BART	Bidirectional and Auto-Regressive Transformer — a model used for text generation and summarization.
DistilBERT	A compact version of BERT used for text classification, here for Bloom’s taxonomy classification.
Bloom’s Taxonomy	A hierarchical model of learning objectives defining levels of cognitive complexity.
Question Paper Pattern	The structure defining marks, number of questions, OR combinations, and module coverage.
GPU	Graphics Processing Unit — required for efficient NLP model execution.
NLP	Natural Language Processing — a branch of AI concerned with understanding and generating human language.
API	Application Programming Interface — not used in this project (offline operation).

2. Literature survey

Automated question-paper generation sits at the intersection of educational theory and modern Natural Language Processing (NLP), integrating principles from pedagogy with advanced machine-learning architectures to create balanced and scalable assessment systems. Classical educational psychology frameworks, most notably Bloom’s Taxonomy (Bloom, 1956) [1], provide the conceptual foundation for defining levels of cognitive complexity—from knowledge recall to higher-order reasoning such as evaluation and creation. Research highlights the importance of distributing questions across these cognitive layers to ensure valid and comprehensive assessment, and modern automated systems strive to embed such pedagogical balance within their generation pipelines.

From a computational perspective, the evolution of text-processing methods has significantly influenced the automation of question generation. Traditional information-retrieval approaches such as TF–IDF and other keyword-based extraction techniques remain valuable due to their interpretability, computational efficiency, and effectiveness in identifying high-relevance topic anchors within syllabus documents and academic texts (Manning et al., 2008). These models enable fast extraction of domain concepts, which is crucial in large-scale educational environments. However, they struggle with semantic nuance and contextual understanding—areas in which neural embedding approaches excel.

To address these limitations, modern systems employ semantic encoders such as Sentence-BERT (SBERT) [3], which generate dense vector representations that capture sentence-level meaning beyond surface lexical overlap (Reimers & Gurevych, 2019). SBERT-based similarity search enables accurate mapping between extracted topics and supporting textual passages in textbooks or lecture material, improving the relevance and specificity of generated questions.

For the question-generation stage, recent literature focuses heavily on sequence-to-sequence transformer models such as T5, BART, and MT5 [4][5], which produce fluent, context-aware questions when fine-tuned on datasets such as SQuAD, RACE, or educational corpora (Lewis et al., 2020; Raffel et al., 2020). These architectures support multiple question types, including short-answer, fill-in-the-blank, and multiple-choice items, and can be conditioned to align with specific difficulty levels or cognitive objectives. To ensure adherence to Bloom’s taxonomy, lightweight transformer-based classifiers such as DistilBERT [6] or RoBERTa are employed to automatically categorize generated questions according to cognitive level, supporting balanced distribution across comprehension, application, analysis, and synthesis categories.

Consequently, contemporary research advocates **hybrid pipeline architectures** that combine:

1. **Efficient topic extraction** (TF–IDF, keyword methods) for coverage and speed,

2. **SBERT-based semantic retrieval** for context-aware content alignment,
3. **Transformer-driven question generation** (T5/BART) for fluency and depth. [2]
4. **Bloom-level classification** for ensuring pedagogical validity,
5. **Human educator review** for quality assurance, ethical integrity, and curricular fit.

Such hybrid systems reflect emerging trends toward intelligent educational technology that not only automates assessment creation but also enhances fairness, standardization, and personalization. This aligns closely with the goals and modules outlined in our SRS document, which emphasize automated yet pedagogically grounded question-paper generation capable of supporting modern digital learning ecosystems.

3. Description

3.1 User Interfaces

The Automated Question Paper Generator (AQPG) will feature a simple, intuitive, and user-friendly graphical interface designed for educators with minimal technical expertise. The system interface will prioritize clarity, guided workflows, and automation to ensure smooth operation.

3.1.1 Interface Design

- The user interface will be built using **React** for easy web-based deployment on local machines.
- The layout will include sequential tabs or pages for:
 1. **Home Page:** Displays system overview and options to start a new session.
 2. **Upload Section:** Allows users to upload syllabus and reference textbooks (PDF/DOCX).
 3. **Configuration Page:** Enables users to define question paper patterns (e.g., number of questions, marks, OR options).
 4. **Generation Page:** Displays real-time progress of question generation, Bloom's classification, and paper assembly.
 5. **Preview and Export Page:** Provides a formatted preview of the generated paper with options to export in DOCX or PDF format.

3.1.2 Input and Output

- **Inputs:**
 - Syllabus document (PDF/DOCX).
 - One or more reference textbooks (PDF/DOCX).
 - Exam pattern configuration (via input fields).
- **Outputs:**
 - Final question paper in DOCX or PDF format.
 - Optional report of Bloom's Taxonomy[1] distribution and topic coverage.

3.1.3 Interface Features

- Interactive buttons for “Upload”, “Generate Questions”, “Classify”, and “Export Paper.”
- Progress bars for showing the stages of processing (Extraction, Mapping, Generation, Classification).
- Error and validation prompts for unsupported file types or missing inputs.
- Optional theme customization for better readability.

3.2 System Interfaces

The system is composed of several interconnected modules that work sequentially and exchange data internally through well-defined interfaces. Each module performs a specific function, and all communicate through structured Python objects or serialized JSON data.

3.2.1 Internal Module Interfaces

Module	Description	Input	Output
Syllabus Ingestion Service (SIS)	Extracts topics and learning objectives from the syllabus.	Syllabus document	List of extracted topics and keywords
Textbook Ingestion Service (TIS)	Parses textbook content and divides it into clean, readable text chunks.	Textbook files	Preprocessed text chunks
Topic–Text Mapping Service (TTS)	Finds relevant textbook sections for each syllabus topic using SBERT.	Syllabus topics, text chunks	Matched topic-chunk pairs
Question Generation Service (QGS)	Generates contextually relevant questions using T5/BART models.	Topic-chunk pairs	Candidate questions
Bloom’s Classification Service (BCS)	Classifies generated questions into Bloom’s levels using DistilBERT.	Generated questions	Questions labeled by Bloom level

Module	Description	Input	Output
Exam Pattern Engine (EPE)	Organizes and selects questions based on user-defined structure.	Classified questions, pattern config	Final structured paper
Output Generator (OG)	Formats and exports question papers.	Final paper data	DOCX/PDF output file

3.2.2 External Interfaces

- **Hardware Interface:** Runs locally on a GPU-enabled laptop/PC (RTX 4050 or higher).
- **Software Interface:** Integrates with Python libraries like PyTorch, Transformers, scikit-learn, pdfminer, and python-docx.
- **File Interface:** Accepts and outputs documents from the local file system; no cloud or API interaction.

3.3 Constraints, Assumptions, and Dependencies

3.3.1 Constraints

1. Hardware Constraints:

- Requires GPU acceleration for NLP model inference.
- Minimum 16 GB RAM and 50 GB disk space.

2. Software Constraints:

- Must operate offline; no use of external APIs.
- All models and datasets must be pre-downloaded.
- Only digital (non-scanned) PDFs and DOCX files are supported.

3. Performance Constraints:

- Paper generation should complete within 2–3 minutes for medium-sized documents.

- Accuracy for topic–text mapping and Bloom’s classification must exceed user-set expectations.

4. **Ethical Constraints:**

- Generated questions must be original and non-plagiarized.
 - Human supervision is required before final usage.
-

3.3.2 Assumptions

- Users will upload clean, structured syllabus and textbook documents.
 - The system will have access to sufficient GPU resources.
 - Users will review and approve the generated question paper before institutional use.
 - Pretrained models (T5, BART, SBERT, DistilBERT)[2][3][4][5] will be fine-tuned on relevant educational data.
-

3.3.3 Dependencies

- **Libraries:** PyTorch, Hugging Face Transformers, scikit-learn, spaCy, NLTK, PyMuPDF, pdfminer.six, python-docx.
- **Datasets:** SQuAD, SciQ, or academic question datasets for model fine-tuning.
- **Hardware:** GPU-enabled machine (NVIDIA RTX 4050 or equivalent).
- **Frameworks:** Streamlit/Flask for frontend interface.

4. Tools Required

4.1 React

Purpose:

React is a popular JavaScript library used for building dynamic and responsive user interfaces. It enables a modular, component-based architecture where each section of the webpage (like input forms, question previews, and upload panels) can be developed and maintained independently.

Why React:

- Offers real-time UI updates using its virtual DOM.
- Highly compatible with APIs, making it easy to connect to a backend built with Node.js or Flask.
- Ideal for single-page applications (SPAs), giving a smooth, app-like experience.

In our system:

React handles the frontend dashboard where instructors can upload syllabus, configure paper patterns, view generated questions, and export the final document.

4.2 Syllabus Extraction / Keywording – Scikit-learn (TF-IDF)

Purpose:

TF-IDF[2] (Term Frequency–Inverse Document Frequency) is a statistical method that identifies important keywords in text by comparing their frequency across documents.

Why Scikit-learn:

- The TfidfVectorizer in scikit-learn is efficient and easy to implement.
- Provides interpretable keyword weights for quick topic extraction.
- Lightweight and scalable for large text corpora.

In our system:

TF-IDF is applied to syllabus or chapter text to find dominant topics or key concepts that guide question generation.

4.3 Textbook Parsing / Chunking – PyMuPDF (fitz library)

Purpose:

PyMuPDF extracts text data from PDF documents efficiently. It helps preprocess source materials (textbooks, lecture notes) so they can be used for further analysis and question generation.

Why PyMuPDF:

- Fast and memory-efficient text extraction.
- Supports image and metadata handling if needed for diagrams or figures.
- Provides positional information, enabling context-aware chunking of pages.

In our system:

It converts syllabus or PDF study materials into plain text chunks that are fed into the question generation model.

4.4 Semantic Topic–Text Mapping – Sentence-Transformers (SBERT)

Purpose:

Sentence-BERT (SBERT)[3] provides semantic embeddings for sentences and paragraphs, allowing comparison based on meaning rather than mere keyword overlap.

Why SBERT:

- Outperforms TF-IDF in understanding contextual similarity.
- Pretrained models like all-MiniLM-L6-v2 are lightweight and effective.
- Works well for topic–content alignment and retrieving relevant passages.

In our system:

SBERT identifies the most relevant text segments from the textbook or notes corresponding to a given topic or learning objective.

4.5 Question Generation – Hugging Face Transformers (T5 Model)

Purpose:

Hugging Face[6] provides powerful NLP models for various text generation tasks. The T5[4] (Text-To-Text Transfer Transformer) model can generate questions from text passages or summaries.

Why Hugging Face T5:

- Versatile: Converts any NLP task into a text-to-text format.
- Pre-trained on QA datasets, producing natural and contextually rich questions.
- Supports easy fine-tuning for domain-specific topics.

In our system:

Given a topic-aligned passage, the T5 model generates short-answer, descriptive, or multiple-choice questions automatically.

4.6 Bloom's Level Classification – DistilBERT

Purpose:

DistilBERT[2][1] is a compressed version of BERT, designed for faster inference while maintaining high accuracy. It classifies generated questions into Bloom's taxonomy levels (like Knowledge, Comprehension, Application, etc.).

Why DistilBERT:

- Efficient for real-time classification on limited hardware.
- Retains 97% of BERT's accuracy at half the size.
- Easily fine-tuned for educational classification tasks.

In our system:

DistilBERT labels each question according to cognitive complexity, ensuring the final paper covers a balanced mix of difficulty levels.

4.7 Exam Pattern Engine / Assembly – Custom Python Rule Engine

Purpose:

This component applies predefined exam rules (e.g., number of questions per unit, mark distribution, difficulty balance) to assemble the final question paper.

Why a Custom Rule Engine:

- Offers flexibility to modify logic per university or subject requirement.
- Integrates seamlessly with other Python-based modules.
- Enables deterministic, reproducible paper structures.

In our system:

It selects questions from the generated pool based on marks, Bloom's level, and topic coverage to finalize the paper.

4.8 Export / Formatting – python-docx & ReportLab

Purpose:

These libraries handle document generation and formatting for the final output[7].

Why python-docx:

- Perfect for creating editable .docx files.

- Supports styling (headings, numbering, spacing) easily.

Why ReportLab:

- Used to generate PDF versions with full layout control.
- Ideal for official printing and digital sharing.

5. User Characteristics

The primary users of this system are educators, academic staff, and examination coordinators. The system is designed to be accessible even to users with limited technical knowledge.

User Type	Description	Technical Skill Level
Teacher / Professor	Responsible for uploading syllabus, textbooks, and defining question paper patterns.	Basic computer literacy
Academic Coordinator	Oversees multiple subjects and verifies quality, syllabus coverage, and Bloom's distribution.	Moderate
Developer / Maintainer	Ensures smooth operation, updates models, and maintains system performance.	Advanced

User Expectations

- Minimal manual input during the question paper creation process.
- Option to preview and modify generated questions before finalization.
- Accurate syllabus coverage and difficulty distribution across modules.

User Environment

- Runs on Windows 10/11 or Ubuntu 22.04 systems.
- Requires only local installation; no internet dependency.
- Provides visual feedback and interactive controls via the user interface.

6. System Features and Requirements

6.1 Functional Requirements

The functional requirements define the primary operations the Automated Question Paper Generator (AQPG) must perform. Each requirement describes what the system should do and how it interacts with users or data.

1 – Upload and Validation of Input Files

- The system shall allow users to upload syllabus and textbook files in PDF, DOCX, or TXT formats.
- Uploaded files shall be validated for file type, readability, and corruption before processing.
- Invalid files shall prompt descriptive error messages.

2 – Syllabus Processing and Keyword Extraction

- The system shall extract and preprocess syllabus text, removing stopwords and irrelevant terms.
- TF-IDF shall identify high-importance keywords and topic phrases representing syllabus content.
- Extracted topics shall be stored as a list of key phrases for later mapping.

3 – Textbook Parsing and Chunking

- The system shall read textbooks using text-extraction libraries (pdfminer / PyMuPDF).
- The text shall be divided into uniform chunks (≈ 200 –300 words) to facilitate semantic matching.
- Non-textual data (tables, figures) shall be ignored or replaced with descriptive tags.

4 – Topic–Text Mapping using SBERT

- Sentence-BERT (SBERT) [3] embeddings shall be generated for both syllabus topics and textbook chunks.
- Cosine similarity shall be computed to identify top-k relevant passages for each topic.
- These matched passages shall be forwarded to the question-generation module.

5 – Question Generation using T5/BART

- The system shall employ a fine-tuned T5 or BART [4],[5] model to generate contextually relevant questions.
- Prompts shall specify question difficulty or marks (e.g., “Generate a 3-mark question”).
- Multiple candidate questions may be generated per passage to ensure diversity.

6 – Bloom’s Taxonomy Classification (DistilBERT)

- Each generated question shall be passed to a trained DistilBERT [6] classifier.
- The classifier shall label the question under one of Bloom’s levels – Remember, Understand, Apply, Analyze, Evaluate, or Create [1].
- These labels shall be used to ensure balanced cognitive distribution in the final paper.

7 – Exam Pattern Definition and Question Assembly

- The user shall define the paper structure (e.g., 8 three-mark and 4 twelve-mark questions with OR options).
- The system shall select appropriate questions across modules according to this pattern.
- Each module shall contribute at least one mandatory question.

8 – Output Formatting and Export

- The final question paper shall be formatted with clear sections, numbering, and institutional details.
- Users shall be able to preview, modify, and export the paper in DOCX or PDF format.
- Exported files shall follow academic document standards.

9 – Extraction of Figures and Tables and Visual Question Generation

- The system shall extract figures, diagrams, charts, and tables from textbook PDFs using PyMuPDF. Each visual element will be stored along with its caption and nearby explanatory text. The Question Generation module shall use this context to create figure-based or table-based questions such as interpretation, inference, and explanation items. The Output Generator shall embed the extracted visual directly into the final DOCX/PDF question paper, and the system shall support generating multiple variations of questions based on the same visual element. This ensures that non-textual academic content is accurately incorporated into the automated question-paper workflow.

6.2 System Diagrams

6.2.1 System Design Diagram

What is a System Diagram?

A system diagram is a visual representation of how different components within a system interact with each other. It illustrates the flow of data, control, or processes between modules, helping stakeholders understand the architecture, dependencies, and high-level operational workflow. System diagrams are commonly used in software engineering to simplify complex systems and communicate structure to developers, designers, and users.

Explanation of the Provided System Diagram

The diagram illustrates the architecture of an Automated Question Paper Generation System, showing how a teacher interacts with back-end services. The user (teacher) uploads syllabus or textbook materials through a web dashboard. The system then processes this input through several connected backend services:

1. Syllabus Ingestion Service (SIS) – Extracts key topics and keywords from syllabus documents.
2. Textbook Ingestion Service (TIS) – Reads and breaks textbook content into structured chunks.
3. Topic–Text Mapping Service (TTS) – Uses semantic matching to link extracted syllabus topics to relevant textbook content.
4. Question Generation Service (QGS) – Generates questions from the matched text using NLP models.
5. Bloom’s Classification Service (BCS) – Classifies generated questions into cognitive levels based on Bloom’s Taxonomy.
6. Output Generator (EPE) – Compiles final formatted question papers and sends them back to the user.

An internal message queue/event bus coordinates communication between services to ensure reliable asynchronous processing.

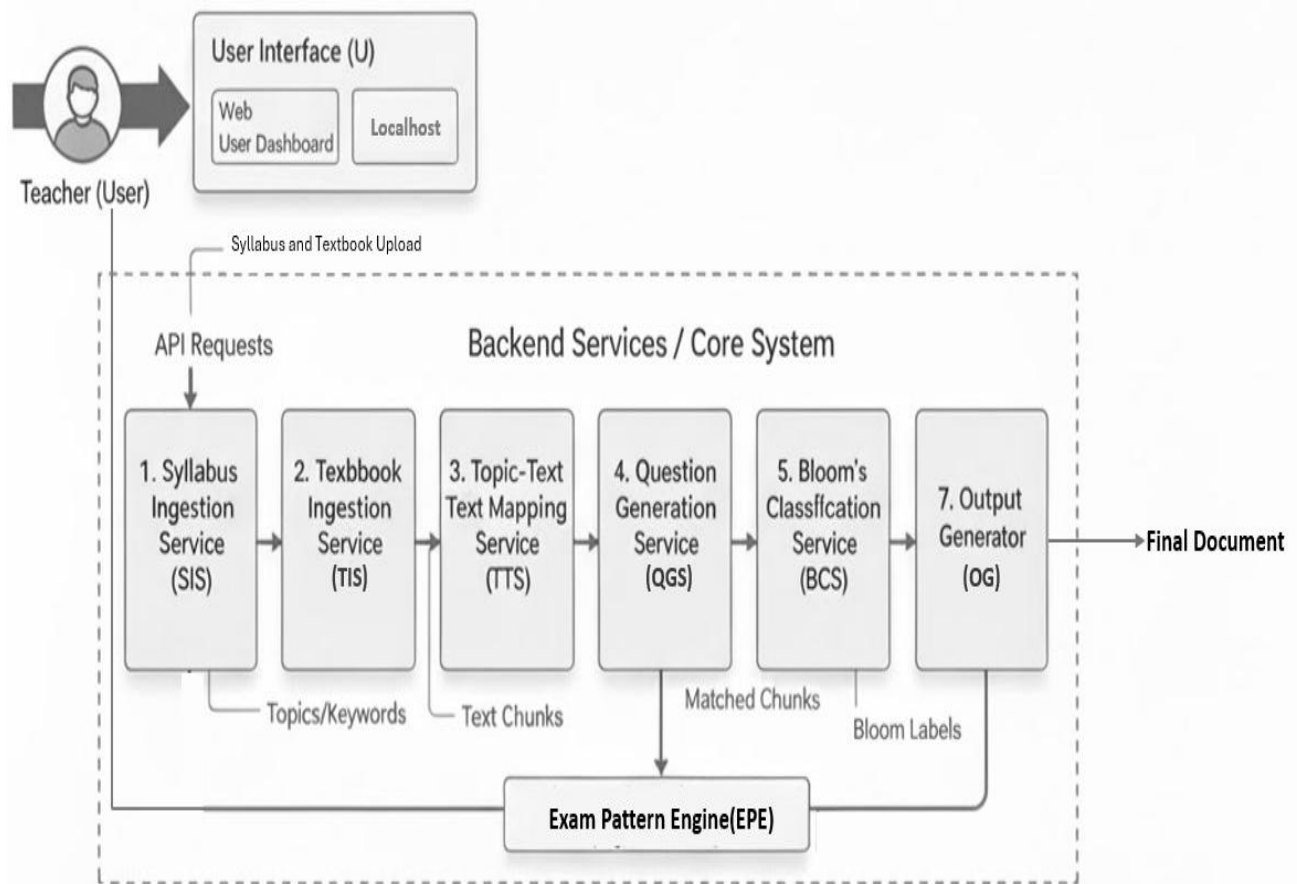


Fig 6.2.1.1: System Design Diagram

1) Syllabus Ingestion System (SIS)

Purpose

- Parse syllabus file(s), extract modules, units, topic lines and any stated learning outcomes/mappings. Produce structured list of topics and candidate keywords.

Pipeline (implementation) :-

1. File extraction

- Use PyMuPDF to extract text.
- If scanned PDFs appear, attempt OCR with Tesseract as fallback (but prefer digital text).

2. Preprocessing

- Normalize whitespace, remove headers/footers (heuristics: repeated text on pages).
- Sentence segmentation with spaCy (language-specific).
- Lowercasing/Unicode normalization depending on needs.

3. Section detection

- Heuristics or simple regex to detect headings like “Module 1”, “Unit II”, or “Syllabus” lines.
- If headings are ambiguous, use layout / font size via PyMuPDF to infer headings.

4. Topic extraction: TF–IDF + phrase detection

- Compute TF–IDF across the syllabus document(s) (scikit-learn TfidfVectorizer).
- Also use n-gram TF–IDF (1–3 grams) to capture multi-word phrases (e.g., “shortest path”).
- For each detected module/section take top-N terms by TF–IDF.

5. Post-filtering

- Remove stopwords, and domain-agnostic terms (“module”, “objective”).
- Optionally use POS tagging to keep nouns/NPs.
- Optionally run KeyBERT (embedding-based keyword ext) to validate TF–IDF top terms.

6. Output

- `topics = [{topic_id, module, text, keywords, context_sentence}]`

Algorithms & specifics

- TF–IDF parameters: `max_df=0.9`, `min_df=1`, `ngram_range=(1,3)`, `max_features=5000`.
- Use phrase merging: detect collocations with PMI or `nlk.collocations` to create multiword tokens.
- Save extracted topics and their representative text to drive retrieval.

Practical tips

- For poorly formatted syllabi, manual mapping UI for teacher to correct topics is essential.
 - Keep confidence scores from TF-IDF (or KeyBERT) so UI can highlight low-confidence topics.
-

2) Textbook Ingestion System (TIS)**Purpose**

- Convert textbooks into clean, chunked passages ready for retrieval. Add metadata (chapter, page, heading).

Pipeline (implementation) :-

1. Extraction
 - PyMuPDF to extract textual content + layout info.
 - If scanned, OCR (Tesseract) with careful post-cleaning.
2. Cleaning
 - Remove headers/footers, page numbers, footnotes heuristically.
 - Normalize special characters and equation markers.
3. Chunking strategy
 - Chunk size: ~150–400 words (experimentally 200–300 words works well).
 - Overlap: 20–40 words overlap between consecutive chunks to keep context for retrieval.
 - Keep chunk metadata: book_id, chapter, page_start, page_end, chunk_id, text.
4. Indexing
 - Build embeddings (SBERT) for chunks and add to vector index (Faiss or simple array with approximate nearest neighbour if small).
 - Also maintain inverted index for TF-term lookup if needed.
5. Storage
 - chunks.jsonl and embeddings.npy (or Faiss index file).

- Keep checksum to verify file integrity.

Practical tips

- Preserve references to original text (for traceability) — we must be able to point a generated question to the source chunk to show the teacher where it came from.
 - For multi-book inputs, record book_priority so retrieval can prefer a primary text if desired.
-

3) SBERT Retrieval Module**Purpose**

- Map syllabus topics (from SIS) to the most relevant textbook chunks using semantic embeddings.

Model & training

- Use a pretrained Sentence-BERT model: recommended all-MiniLM-L6-v2 (fast) or paraphrase-mpnet-base-v2 (higher quality) depending on VRAM.
- Fine-tuning (recommended) on domain-specific pairs:
 - Create training pairs: (topic_text, chunk_text) labeled 1 if chunk covers topic, 0 otherwise.
 - Loss: Contrastive Loss or MultipleNegativesRankingLoss (from sentence-transformers) to pull relevant pairs together and push irrelevant ones apart.
 - Batch size: 16–32 (depends on GPU). Learning rate: 2e-5–5e-5. Epochs: 1–3 for domain adaptation.
 - Use hard negatives by sampling chunks from same chapter but unrelated to topic (improves discrimination).
- Save dimension (e.g., 384 or 768) embedding vector for chunks and topics.

Retrieval algorithm

1. Embed: topic → vector; chunk embeddings precomputed.
2. Compute similarities: cosine similarity between topic vector and chunk vectors.
3. Top-k retrieval: retrieve top-k (k=3–10, typically 5) chunks per topic.
4. Filtering:

- apply threshold (cosine ≥ 0.65 as a heuristic; tune on dev set).
 - remove near-duplicates by measuring chunk-chunk similarity to ensure diversity.
5. Return: [(chunk_id, similarity_score), ...] to QG module.

Implementation details

- Use Faiss Index for large corpora (IVF/PQ for larger datasets). For 1–10k chunks, brute-force cosine with optimized matrix multiplication may be fine.
- Embedding normalization: L2-normalize embeddings for cosine similarity.

Evaluation

- Precision@k and recall@k using a labeled validation set (topic $\rightarrow$ gold chunks).
 - Human evaluation: teachers validate retrieved chunks.
-

4) Question Generation Module (T5 or BART)

Purpose

- Given a chunk (and its associated topic), generate candidate questions with controlled difficulty/marks.

Model choices

- T5 (text-to-text) or BART (seq2seq denoising). Planning to use T5-base or BART-base for RTX 4050.
- Optionally use flan-t5 variants if available for better instruction following (but resource heavier).

Training / Fine-tuning

1. Dataset
 - Public QA datasets (SQuAD v1/v2), NewsQA, NaturalQuestions, SciQ.
 - Create domain-specific fine-tuning dataset by aligning retrieved chunks to human-written questions where possible (supervised).
2. Input format (prompting)
 - Standardized prompt pattern. Examples:

- “generate_question: [difficulty=3] context: <chunk_text> topic: <topic>”
 - “generate_question: [marks=12, level=analyze] context: <chunk>”
 - T5 treats that as text → text.
3. Training objective
- Teacher-forcing cross-entropy loss between model outputs and reference questions.
 - Optionally use label smoothing (epsilon=0.1) to improve generalization.
4. Hyperparameters
- batch_size: 4–16 (GPU-limited), learning rate 1e-5–5e-5, epochs 3–5 for domain adaptation.
 - use AdamW, weight decay 0.01.
 - use mixed precision (fp16) to speed up.
5. Decoding strategies
- For generation, use beam search (beam=3–5) for high-quality outputs or nucleus sampling (top_p = 0.9) for diversity.
 - Use length penalties and min/max length constraints tuned by marks:
 - 3-mark → 8–20 tokens, 12-mark → 30–120 tokens.
6. Post-generation filtering
- Remove duplicated questions.
 - Remove hallucinations/facts outside text: perform a simple string-match check: require that the answer (or key phrase) exists in the chunk, or require presence of topic keywords.
 - Use heuristics to ensure question type matches marks (short factual vs analytical).

Practical techniques

- RAG-like grounding: always pass the retrieved chunk explicitly into the input so the model is grounded and less likely to hallucinate.
- Candidate generation: generate N candidates (N=3–7) and then rank via a scoring function (language-model log-probability, relevance to topic).

- Reward modeling: later you can implement a small reranker (BERT-based) trained to score question appropriateness.

Evaluation

- Use BLEU / ROUGE for automatic comparisons (limited usefulness). More important:
 - Human evaluation: teachers rate relevance, difficulty appropriateness, and answerability.
 - Answerability: check whether a valid short answer can be produced from chunk text (e.g., run an extractive QA to find an answer span for the generated question — if no span, mark as low quality).
-

5) DistilBERT Bloom Classification Module

Purpose

- Label each generated question with one of Bloom's taxonomy levels (Remember, Understand, Apply, Analyze, Evaluate, Create) — used to balance the paper.

Model & training

- Use distilbert-base-uncased with a classification head (6-way softmax).
- Dataset
 - Ideally we need labeled dataset of questions → Bloom level. If unavailable, we will build one:
 - Manually label ~1,000–2,000 questions across levels (small but effective).
 - Augment with synthetic labeling rules (heuristics for Remember vs Create), then clean with human review.
 - Alternatively, we could annotate public QA data with heuristics (short factual Qs → Remember; long explanatory Qs → Analyze/Create).
- Training
 - Loss: Cross-entropy.
 - Handle class imbalance with weighted loss or oversampling minority classes.
 - Batch size: 16–32; LR $2e-5$; epochs 3–4.

- Evaluate with accuracy, macro-F1 (important when classes imbalanced).
- Inference
 - For each question, get logits $\rightarrow$ softmax $\rightarrow$ label + confidence score. Keep top-2 labels if confidence low (teacher review).

Practical heuristics and features

- Use features beyond text: generation prompt used (if you asked for an Analyze-level question) can be a prior.
 - Use question length, presence of action verbs (“compare”, “design”, “analyze”) as signals.
 - Optionally train a small rule-based fallback for obvious cases.
-

6) Exam Pattern Engine (EPE)

Purpose

- Given a set of candidate questions with Bloom labels, module/topic tags, and marks, assemble a final paper that meets user-specified constraints (marks distribution, module coverage, OR logic, per-module question count).

Problem formulation

- This is a constrained selection problem. Can be solved as:
 - Greedy heuristic (fast): pick questions to satisfy constraints iteratively.
 - Integer Linear Programming (ILP) (optimal): formulate selection as ILP and solve with pulp / ortools (very feasible for exam-sized selection).
 - Constraint programming if very complex rules exist.

Example ILP formulation

- Variables: $x_i \in \{0,1\}$ for each candidate question i (1 if selected).
- Constraints:
 - $\sum x_i * \text{marks}_i = \text{TotalMarks}$.
 - For each module m : $\min_m \leq \sum_{i \text{ in module } m} x_i \leq \max_m$.
 - For each Bloom level b : $\min_b \leq \sum_{i \text{ with level } b} x_i \leq \max_b$.

- OR pairs: For each OR-pair (q_a, q_b), ensure either $x_a + x_b \geq 1$? Typically we need to produce paired questions with OR format — implement by selecting pairs and formatting accordingly.
- Ensure uniqueness: only one question per chunk or enforce chunk coverage constraints.
- Objective: maximize total quality score (sum of $x_i * \text{quality}_i$) where quality_i is derived from generation confidence, SBERT similarity, novelty score.
- Solve with pulp/CBC (open-source) quickly for <200 candidate items.

Implementation details

- Pairing OR questions: If pattern needs OR within same module, EPE needs to select pairs (q_1 OR q_2) where both belong to same module and have similar marks. Represent pairs as new items in selection space or enforce pair constraint.
- Diversity: penalize selecting two very similar questions (chunk-chunk similarity > 0.85). Add constraints or objective penalties.
- Fallback: if ILP infeasible, relax constraints (increase allowed module range) and notify user.

Output

- Ordered list of questions with section headings, numbering, marks, OR formatting, and references to source chunk (for teacher traceability).

7) Output Generator (OG)

Purpose

- Convert assembled question list to a nicely formatted DOCX or PDF that conforms to exam standards.

Implementation

- DOCX: python-docx for structured Word output.
 - Template docx with placeholders for header (university name, subject, code, date).
 - Use styles: Heading 1 for Section, Normal for question text.

- OR formatting: use table or two-column layout like:
- Q5. (a) ... (b) OR (attempt either (a) or (b))
- Include question metadata as hidden comments or a separate appendix: question_id, module, chunk_ref, bloom_level, confidence.
- PDF: either export DOCX → PDF (Word conversion) or generate with reportlab for direct PDF generation.
- Answer Key: optional separate doc with model answers (if you implement extractive QA or generate answers).
- Versioning: generate multiple sets (A/B) using paraphrase/alternate generation; shuffle within constraints to avoid duplicate answer keys.
- Accessibility: support readable fonts and avoid images unless necessary.

Practical details

- Pagination, page breaks, and consistent numbering are essential.
- Provide teacher the option to edit in-app before final export (preview + inline edit).
- Produce a small coverage report (module coverage %, Bloom distribution histogram) as separate appendix.

8) User Interface (UI) & Orchestration

Purpose

- Enable teacher / coordinator interactions: uploads, configuration, review, edit, final export.

Architecture

- Prototype: Streamlit (fast to build) or simple Flask + minimal React UI.
- Pages / Flow:
 1. Upload: syllabus & textbook(s). Validate and show parsed topics.
 2. Configure: exam pattern builder (marks, question counts, OR logic), module constraints, versioning options.

3. Generate: shows progress bars for pipeline stages (ingestion → retrieval → generation → classification → assembly).
 4. Review: preview paper, per-question metadata, allow accept/reject/replace question; manual edits allowed.
 5. Export: produces DOCX/PDF and coverage report.
- Backend: orchestrate pipeline tasks with job manager (simple queue + worker). Use subprocesses for long-running operations.
 - Logs & Monitoring: show logs, allow retriggering specific stages (generate again with different seed/prompt).
 - User settings: ability to set beam size, generation temperature, top-k retrieval count — but hide advanced settings in “advanced” section.

Failures & UX

- Show meaningful messages on failure (e.g., “no relevant chunks found for Topic X — try adding more textbooks or adjust TF-IDF thresholds”).
- Allow the user to correct topics in UI (rename / merge / reassign) and re-run retrieval.

Integration & End-to-end Workflow (summary)

1. Teacher uploads syllabus & textbook → SIS identifies topics.
 2. TIS splits textbooks into chunks, indexes and computes chunk embeddings.
 3. SBERT retrieval: for each topic, return top-k relevant chunks.
 4. QG (T5/BART): for each chunk, generate N candidate questions, conditioned on marks/difficulty prompt.
 5. DistilBERT classifier: label each question with Bloom level + confidence.
 6. EPE runs (ILP/greedy) to select a set satisfying constraints and maximizing quality score.
 7. OG formats and exports paper; teacher reviews and finalizes.
-

Training & Validation Strategy (practical plan)

1. Collect data

- Public QG datasets and university past papers (if available).
- Manually label a small gold set for evaluation (topic→chunk, question→Bloom).

2. Pretrain / Fine-tune in stages:

- Fine-tune SBERT on topic-chunk pairs (contrastive loss).
- Fine-tune T5/BART on QA QG pairs; warm-up with SQuAD then domain-adapt using mapped topic-chunk→question pairs.
- Train DistilBERT classifier on Bloom-labeled dataset.

3. Validation

- SBERT: Precision@k and recall@k on held-out topic-chunk pairs.
- QG: Human ratings; automatic metrics (ROUGE/BLEU) for debug but not final judge.
- DistilBERT: macro-F1, confusion matrix.
- End-to-end: sample several papers and have domain-experts rate coverage, correctness, and difficulty balance.

4. Hyperparameters

- Keep experiments tracked (We use wandb locally or record CSV of runs).
- Typical values:
 - SBERT: lr 2e-5, bs 32, epochs 2.
 - T5/BART: lr 3e-5–5e-5, bs 4–8, epochs 3.
 - DistilBERT: lr 2e-5, bs 16, epochs 3.

5. Iterate

- Use teacher feedback to improve prompts, chunking, and retrieval thresholds.
 - Add data via teacher corrections into training corpora (active learning loop).
-

6.2.2 UseCase Diagrams

Usecase Diagram for user creating the Question Paper

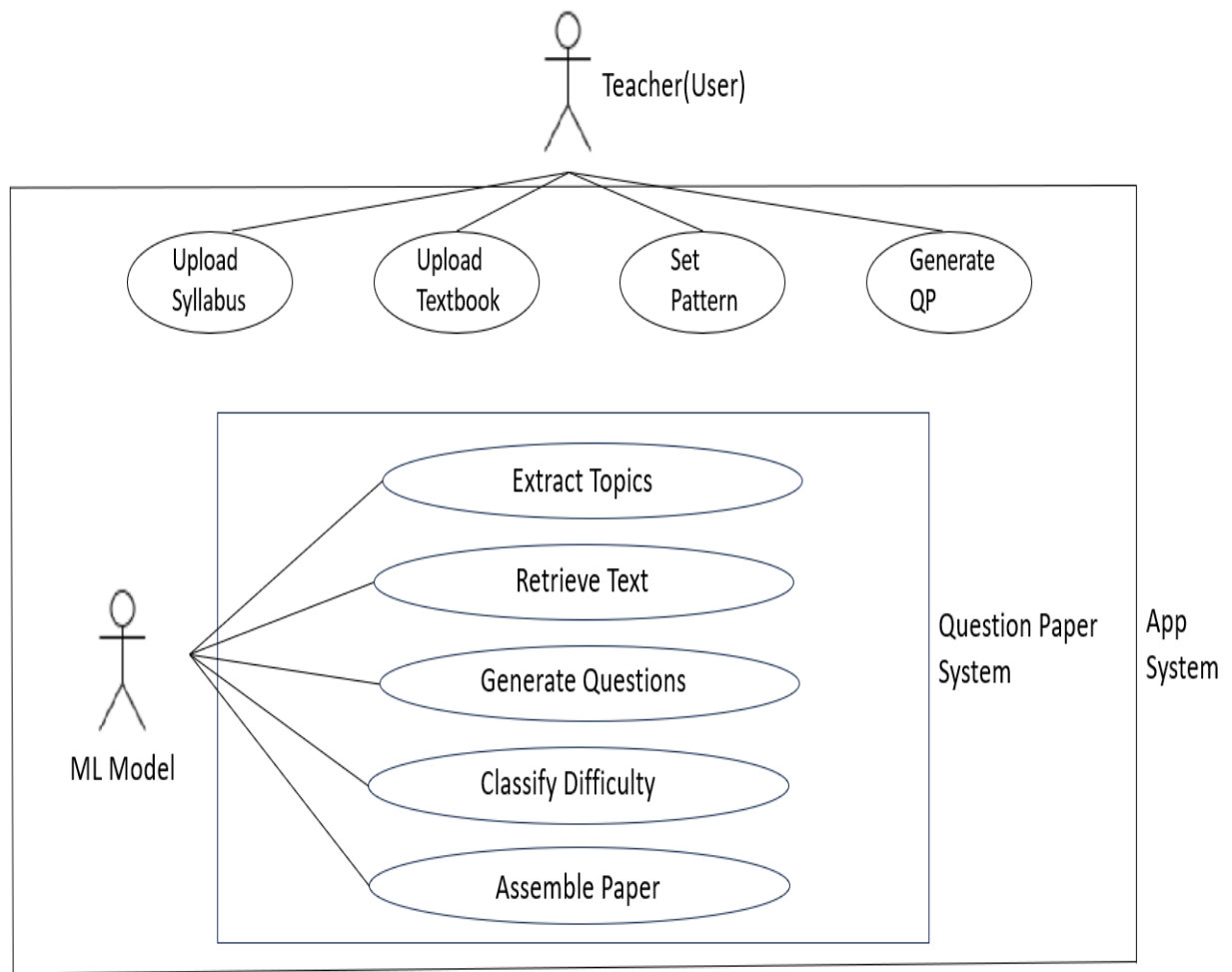


Fig 6.2.2.1: Usecase Diagram for Question Paper Creation

Teacher (User): The primary actor who interacts with the system to create question papers.

Use Cases (Teacher Actions):

- **Upload Syllabus:** The teacher uploads the course syllabus to the system. This likely informs the topics from which questions will be drawn.
- **Upload Textbook:** The teacher uploads the relevant textbook or study material. This is the source content for question generation.
- **Set Pattern:** The teacher defines the structure and format of the question paper (e.g., number of questions, mark distribution, types of questions).

- **Generate QP (Question Paper):** The teacher initiates the process of generating a question paper based on the uploaded syllabus, textbook, and set pattern.

Internal System Processes (Question Paper System):

- **Extract Topics:** The system analyzes the uploaded syllabus and textbook to identify key topics and concepts.
- **Retrieve Text:** The system retrieves relevant text segments from the uploaded textbook based on the extracted topics.
- **Generate Questions:** The system creates questions based on the retrieved text and the defined question paper pattern.
- **Classify Difficulty:** The system assigns a difficulty level to each generated question.
- **Assemble Paper:** The system compiles the generated and classified questions into a complete question paper according to the set pattern.

Output:

- **Question Paper PDF:** The final output, a downloadable PDF document of the generated question paper.

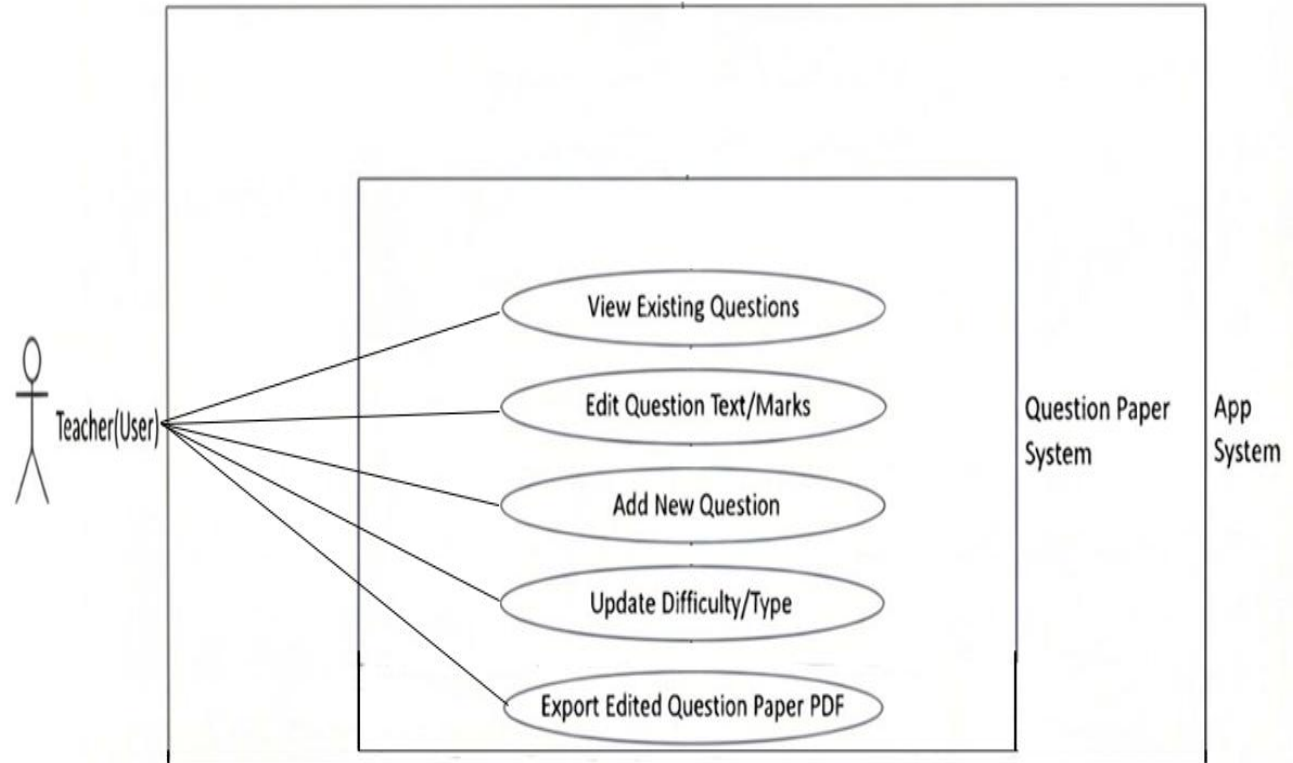
Usecase Diagram for user modifying the Question Paper

Fig 6.2.2.2 : Usecase Diagram for Question Paper Modification

Teacher (User): The primary actor who interacts with the system to modify existing question papers.

Use Cases (Teacher Actions):

- **Open Question Paper for Editing:** The teacher selects an existing question paper within the system to make modifications. This initiates the editing process.

Internal System Processes (Question Paper Editing System):

- **View Existing Questions:** The system displays all questions currently in the selected question paper, allowing the teacher to review them.
- **Edit Question Text/Marks:** The teacher can modify the wording of a question, change its assigned marks, or correct any errors.

- **Add New Question:** The teacher can insert new questions into the question paper. This might involve manually typing a question or selecting from a question bank if available.
- **Update Difficulty/Type:** The teacher can change the difficulty level or question type (e.g., multiple-choice, short answer) of an existing question.

Output:

- **Edited Question Paper PDF:** The final output, a downloadable PDF document of the modified question paper, reflecting all the changes made by the teacher.

6.2.3 Sequence Diagram

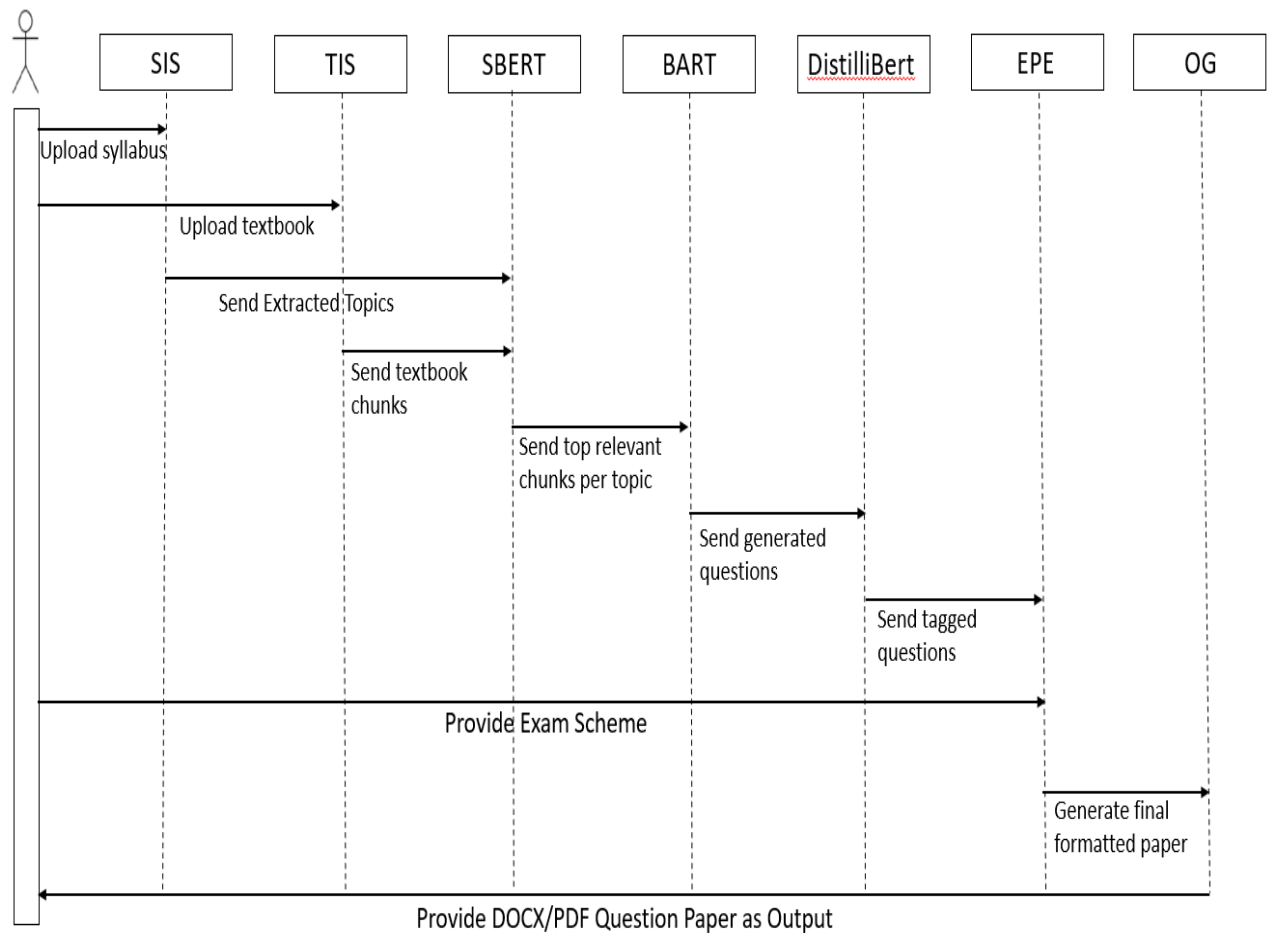


Fig 6.2.3.1 : Sequence Diagram

This Sequence Diagram illustrates the temporal order of interactions between the user, the application interface components (SIS, TIS, EPE, OG), and the underlying data/model layers (SBERT, BART, DistillBert). The sequence starts with the user providing input via Upload syllabus and Upload textbook, followed by the system sending messages like Send Extracted Topics, Send textbook chunks, Send generated questions, and Provide Exam Scheme. The process culminates in the system executing Generate final formatted paper and providing the final DOCX/PDF Question Paper as Output.

6.2.4 Activity Diagram

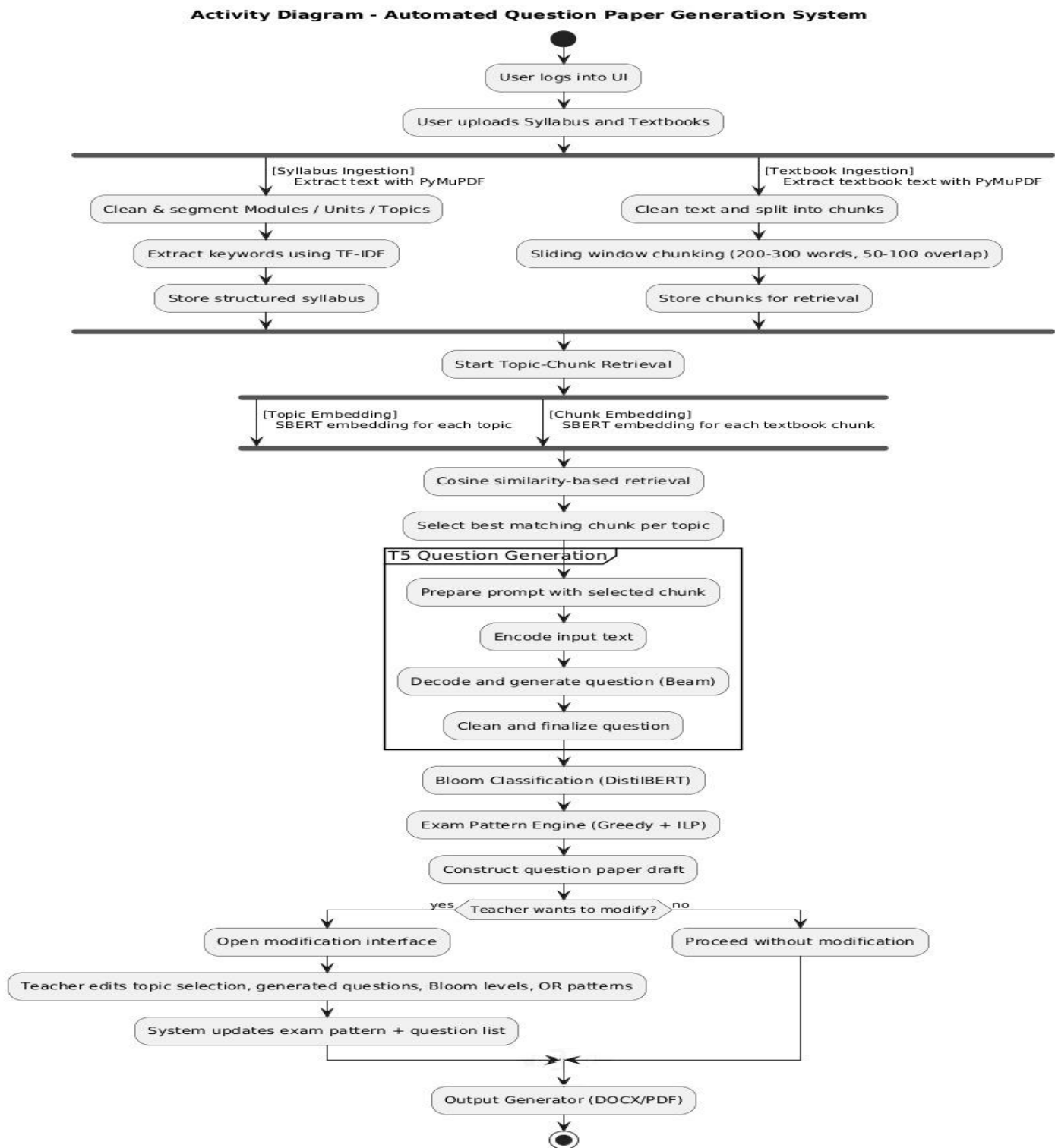


Fig 6.2.4.1 : Activity Diagram

This models the detailed workflow for the automatic question paper generation process, starting with the Teacher (User) initiating the system. The flow begins with the teacher actions of Start and Upload syllabus and Upload reference textbooks. The subsequent system steps include Extract topics , Parse textbooks , Retrieve relevant text , Generate question , and Classify difficulty. The process concludes with applying the exam pattern and Assemble paper to yield the Downloadable Paper output

6.2.5 Gantt Chart

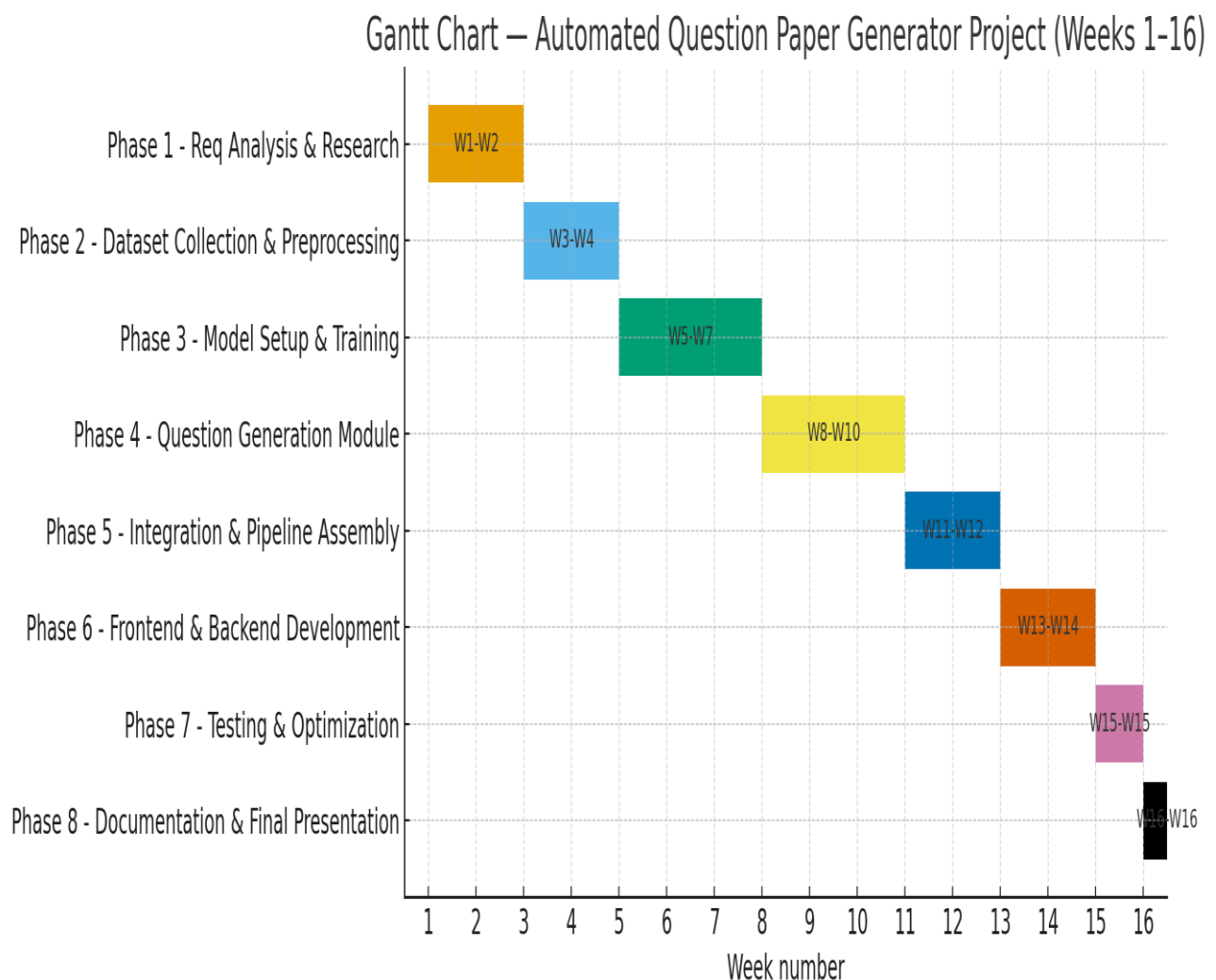


Fig 6.2.5.1: Gantt Chart

Phase	Tasks	Duration	Timeline
Phase 1 – Requirement Analysis & Research	Study problem domain, collect references, finalize scope, identify models (TF-IDF, SBERT, T5, DistilBERT), prepare SRS & feasibility report	2 weeks	Week 1–2
Phase 2 – Dataset Collection & Preprocessing	Collect syllabus documents and textbooks, prepare topic-text pairs, clean and tokenize data, build labeled dataset for Bloom’s taxonomy	2 weeks	Week 3–4
Phase 3 – Model Setup & Training	Install dependencies (PyTorch, Transformers), prepare TF-IDF extractor, fine-tune SBERT for topic-text mapping, fine-tune DistilBERT for Bloom classification	3 weeks	Week 5–7
Phase 4 – Question Generation Module	Fine-tune T5/BART on question generation datasets (SQuAD/SciQ), implement prompt-based difficulty control	3 weeks	Week 8–10
Phase 5 – Integration & Pipeline Assembly	Integrate all models (TF-IDF → SBERT → T5 → DistilBERT → Rule Engine), test full workflow	2 weeks	Week 11–12
Phase 6 – Frontend & Backend Development	Build Streamlit/Flask interface for uploading syllabus, setting paper patterns, generating outputs	2 weeks	Week 13–14
Phase 7 – Testing & Optimization	Evaluate accuracy, Bloom classification balance, optimize runtime, fix UI issues	1 week	Week 15
Phase 8 – Documentation & Final Presentation	Prepare final report, PPT, project demo video, and presentation materials	1 week	Week 16

6.3 External Interface Requirements

6.3.1 Hardware Interfaces

- GPU-enabled computer (NVIDIA RTX 4050 or equivalent).
- Minimum 16 GB RAM and 50 GB storage.
- 1080p display for UI rendering.

6.3.2 Software Interfaces

- **Operating Systems:** Windows 10/11 or Ubuntu 22.04.
- **Programming Language:** Python 3.10+.
- **Frameworks & Libraries:** PyTorch, Transformers (Hugging Face), scikit-learn, spaCy, NLTK, PyMuPDF, pdfminer, python-docx, reportlab.
- **Frontend:** React web framework.

6.3.3 User Interfaces

- Simple GUI with buttons for Upload, Generate, Preview, Export.
- Real-time progress bar and error notifications.
- Support for light and dark themes.

6.4 Logical Database Requirements

Although the system primarily operates on files, minimal data persistence is required.

- **Local Storage:** Temporary storage for uploaded files and generated outputs.
- **Metadata Files:** Maintain JSON records of extracted topics, matched chunks, and Bloom's classification results.
- **Directory Structure:**
 - ✓ /input_files
 - ✓ /processed_data
 - ✓ /generated_questions

- ✓ /exported_papers
 - All temporary data shall be deleted after session completion to maintain privacy.
-

6.5 Non-Functional Requirements

6.5.1 Performance

- Average paper generation time ≤ 2 minutes for ≤ 100 -page inputs.
- Memory usage optimized to avoid system freeze during model inference.

6.5.2 Reliability

- The system shall recover from interrupted sessions without data loss.
- Logs shall be maintained for debugging and audit purposes.

6.5.3 Usability

- User operations require no technical expertise in NLP.
- Simple step-by-step workflow ensures smooth operation.
- Comprehensive help section and error messages.

6.5.4 Portability

- Compatible across Windows and Linux.
- Installable through requirements.txt and launchable via single command.

6.5.5 Security

- All data processed offline – no external API calls.
- Automatic deletion of temporary files post-session.

6.5.6 Maintainability and Scalability

- Modular architecture allows individual component upgrades (TF-IDF $\rightarrow$ KeyBERT, T5 $\rightarrow$ Flan-T5 etc.).
- Documentation and version control maintained via GitHub repository.

7. Conclusion

The Automated Question Paper Generation System successfully integrates pedagogical principles with modern NLP and machine-learning techniques to automate the creation of balanced and syllabus-aligned academic question papers. By incorporating Bloom's Taxonomy as the theoretical foundation for cognitive diversity [1], transformer-based models for semantic understanding and question generation [2],[4],[5], and SBERT embeddings for accurate topic-text mapping [3], the system overcomes the limitations of traditional manual paper-setting methods. Topic extraction was enabled through TF-IDF for interpretable keyword identification, while DistilBERT supported Bloom-level classification to ensure structured complexity across generated questions. The workflow described in the SRS and represented in system design diagrams (page 18 diagram section) demonstrates how each referenced technique has been integrated into a hybrid, practical pipeline that supports real-world academic examination needs.

In future development phases, the system can be extended through additional capabilities such as: (i) support for multilingual content and regional language syllabi, (ii) integration of automated answer-key generation through extractive QA models, (iii) reinforcement learning-based feedback loops to continuously improve question quality using teacher ratings, and (iv) cloud deployment for collaborative use across institutions. Further enhancement may include upgrading to larger instruction-tuned models such as FLAN-T5 and improved semantic retrievers like ColBERT or RAG-based architectures for more accurate context grounding. With these improvements, the system could evolve from a semi-automated tool into a fully adaptive assessment intelligence platform, providing high-quality, pedagogically aligned questions with minimal human intervention.

8. References

The development and conceptualization of this system are guided by the following references:

1. **Bloom, B. S. (1956).** *Taxonomy of Educational Objectives: The Classification of Educational Goals.*
2. **Vaswani, A. et al. (2017).** *Attention Is All You Need.* Proceedings of NeurIPS 2017.
3. **Reimers, N., & Gurevych, I. (2019).** *Sentence-BERT: Sentence Embeddings using Siamese BERT Networks.*
4. **Raffel, C. et al. (2020).** *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer (T5).* Journal of Machine Learning Research.
5. **Lewis, M. et al. (2020).** *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation.*
6. **Hugging Face Transformers Documentation.** <https://huggingface.co/docs/transformers>
7. **Python Libraries:** spaCy, NLTK, scikit-learn, PyTorch, and pdfminer.six official documentation.
8. University and institutional exam guidelines for structuring question papers.